

Vecteurs

Objectifs d'apprentissage

- Apprendre les différentes structures de données dans R
- Comment travailler avec des vecteurs dans R

Les données appartiennent à différentes *classes*, comme expliqué dans le module précédent, et peuvent être organisées en différentes **structures**.

Jusqu'à présent, nous n'avons traité que des variables contenant une seule valeur, mais la véritable valeur de R réside dans l'attribution de *ensembles complets* de données à une variable.

La structure de données la plus simple dans R est un **vecteur**. Un vecteur est simplement un ensemble de valeurs. Un vecteur peut contenir une seule valeur, comme nous l'avons vu jusqu'à présent, ou plusieurs millions de valeurs.

Déclaration et utilisation des vecteurs

Pour créer un vecteur dans R, utilisez la fonction `c()`, qui est l'abréviation de "concatenate" (concaténer).

```
x <- c(5,6,7,8)
x
## [1] 5 6 7 8
```

Chaque fois que vous utilisez la fonction `c()`, vous dites à R : "Hé, prépare-toi. Je vais te donner plusieurs valeurs à la fois. ". Hum hum.

Vous pouvez utiliser la fonction `c()` pour concaténer deux vecteurs :

```
x <- c(5,6,7,8)
y <- c(9,10,11,12)
z <- c(x,y)
z
## [1] 5 6 7 8 9 10 11 12
```

Vous pouvez également utiliser `c()` pour ajouter des valeurs à un vecteur :

```
x <- c(5,6,7,8)
x <- c(x,9)
x
## [1] 5 6 7 8 9
```

Vous pouvez également soumettre des vecteurs à des tests logiques :

```
x <- c(1,2,3,4,5)
4 == x
## [1] FALSE FALSE FALSE TRUE FALSE
```

Cette commande demande à R de vous indiquer si chaque élément de `x` est égal à 4.

Vous pouvez créer des vecteurs de n'importe quelle classe de données (c'est-à-dire de n'importe quel type de données).

```
x <- c("Greta","Nika","Crispin")
x
## [1] "Greta" "Nika" "Crispin"
```

```
y <- c(TRUE,TRUE,FALSE)
y
## [1] TRUE TRUE FALSE
```

Notez que toutes les valeurs d'un vecteur doivent être de la même classe. Vous ne pouvez pas combiner des nombres et des caractères dans le même vecteur. Si vous le faisiez, R essaierait de convertir les nombres en caractères. Par exemple :

```
x <- 4
y <- "6"
z <- c(x,y)
z
## [1] "4" "6"
```

Mathématiques avec deux vecteurs

Lorsque deux vecteurs ont la même longueur, vous pouvez effectuer des opérations arithmétiques avec eux :

```
x <- c(5,6,7,8)
y <- c(9,10,11,12)
x + y
## [1] 14 16 18 20
```

```
x - y
## [1] -4 -4 -4 -4
```

```
x * y
## [1] 45 60 77 96
```

```
x / y
## [1] 0.5555556 0.6000000 0.6363636 0.6666667
```

Que se passe-t-il lorsque deux vecteurs n'ont pas la même longueur ?

Eh bien, cela dépend. Si l'un des vecteurs a une longueur de 1 (c'est-à-dire un seul nombre), alors tout fonctionne généralement bien.

```
x <- 5
y <- c(1,2,3,4,5,6,7,8,10)
x + y
## [1] 6 7 8 9 10 11 12 13 15
```

Dans cette commande, l'élément unique de `x` est ajouté à chaque élément de `y`.

Un autre exemple, que vous avez déjà vu ci-dessus :

```
a <- c(1,2,3,4,5)
b <- 4
a == b
## [1] FALSE FALSE FALSE TRUE FALSE
```

Dans cette commande, l'élément unique de `b` est comparé à chaque élément de `a`.

Cependant, lorsque les deux vecteurs contiennent plusieurs valeurs mais ne sont pas de même longueur, **attention** : des anomalies peuvent se produire. En effet, `R` commencera à recycler le vecteur le plus court :

```
a <- c(1,2,3,4,5)
b <- c(3,4)
a + b
## Warning in a + b: longer object length is not a multiple of shorter object
## length
## [1] 4 6 6 8 8
```

Comme l'indique cet avertissement, cela n'a pas beaucoup de sens. La commande s'exécutera quand même, mais ne vous fiez pas au résultat.

Fonctions pour manipuler les vecteurs

Nous allons énumérer une série de fonctions essentielles pour travailler avec des vecteurs. Considérez cela comme une boîte à outils. Chaque outil a un usage spécifique et une valeur limitée : vous ne pouvez pas construire une maison avec un simple marteau. Mais lorsque vous apprenez à utiliser tous les outils de votre boîte à outils *ensemble*, vous pouvez construire presque tout. Cependant, vous devez d'abord savoir utiliser chaque outil individuellement.

`length()` vous indique le nombre d'éléments dans un vecteur :

```
x <- c(5,6)
length(x)
## [1] 2
```

```
y <- c(9,10,11,12)
length(y)
## [1] 4
```

Le **symbole deux-points** : crée un vecteur contenant tous les nombres entiers compris entre une valeur minimale et une valeur maximale :

```
x <- 1:10
x
## [1] 1 2 3 4 5 6 7 8 9 10
```

`seq()` vous permet de construire un vecteur en utilisant une *séquence* de valeurs espacées de manière régulière entre une valeur minimale et une valeur maximale :

```
seq(0,100,length=11)
## [1] 0 10 20 30 40 50 60 70 80 90 100
```

Dans cette commande, vous demandez à R de vous donner une séquence de valeurs comprises entre 0 et 100, et vous voulez que la longueur de ce vecteur soit 11. R calcule alors l'espacement nécessaire entre chaque valeur pour que cela soit possible.

Vous pouvez également prescrire l'intervalle entre les valeurs au lieu de la longueur :

```
seq(0,100,by=7)
## [1] 0 7 14 21 28 35 42 49 56 63 70 77 84 91 98
```

`rep()` vous permet de répéter une seule valeur un nombre spécifié de fois :

```
rep(" Hey! ",times=5)
## [1] " Hey! " " Hey! " " Hey! " " Hey! " " Hey! "
```

Vous pouvez également utiliser `rep()` pour répéter chaque élément d'un vecteur un nombre défini de fois :

```
rep(c(" Hey! ", " Wohoo! "),each=3)
## [1] " Hey! " " Hey! " " Hey! " " Wohoo! " " Wohoo! " " Wohoo! "
```

`head()` et `tail()` peuvent être utilisés pour récupérer respectivement les 6 premiers ou les 6 derniers éléments d'un vecteur.

```
x <- 1:1000
head(x)
## [1] 1 2 3 4 5 6
tail(x)
## [1] 995 996 997 998 999 1000
```

Vous pouvez également ajuster le nombre d'éléments à renvoyer :

```
head(x,2)
## [1] 1 2
tail(x,10)
## [1] 991 992 993 994 995 996 997 998 999 1000
```

`sort()` vous permet de trier un vecteur de la plus petite à la plus grande valeur :

```
x <- c(4,8,1,6,9,2,7,5,3)
sort(x)
## [1] 1 2 3 4 5 6 7 8 9
```

rev() vous permet d'inverser l'ordre des éléments d'un vecteur :

```
x <- c(4,8,1,6,9,2,7,5,3)
rev(x)
## [1] 3 5 7 2 9 6 1 8 4
```

```
rev(sort(x))
## [1] 9 8 7 6 5 4 3 2 1
```

min() et **max()** vous permettent de trouver la plus petite et la plus grande valeur d'un vecteur.

```
min(x)
## [1] 1
```

```
max(x)
## [1] 9
```

which() vous permet de demander : " Pour quels éléments d'un vecteur l'affirmation suivante est-elle vraie ? "

```
x <- 1:10
which(x==4)
## [1] 4
```

Si aucune valeur du vecteur ne remplit la condition, un vecteur de longueur zéro sera renvoyé :

```
x <- 1:10
which(x == 11)
## integer(0)
```

which.min() et **which.max()** vous indiquent respectivement quel élément est le plus petit et le plus grand dans le vecteur :

```
which.min(x)
## [1] 1
```

```
which.max(x)
## [1] 10
```

%in% est un opérateur pratique qui vous permet de demander si une valeur apparaît *dans* un vecteur :

```
x <- 1:10
4 %in% x
## [1] TRUE
```

```
11 %in% x
## [1] FALSE
```

`is.na()` permet de demander si un vecteur contient des valeurs manquantes, corrompues ou erronées. Dans R, ces valeurs sont désignées par l'expression `NA`. Lorsque vous voyez `NA`, considérez que R vous dit : "Non, non ! Pas disponible !"

```
x <- c(3,5,7,NA,9,4)
is.na(x)
## [1] FALSE FALSE FALSE TRUE FALSE FALSE
```

Cette fonction parcourt chaque élément du vecteur `x` et vous indique si cet élément est `NA`.

Sous-ensembles de vecteurs

Comme vous serez amené à travailler avec des vecteurs contenant des milliers de points de données, il sera utile de disposer d'outils permettant de créer des sous-ensembles, c'est-à-dire de n'examiner que quelques éléments sélectionnés à la fois.

Vous pouvez sous-diviser un vecteur à l'aide de crochets `[ ]`. Chaque fois que vous utilisez des crochets, vous dites à R : "Bonjour, je veux certains nombres, mais *pas tous* : seulement certains d'entre eux."

```
x <- 50:100
x[10]
## [1] 59
```

Cette commande demande à R de renvoyer le 10^e élément du vecteur `x`.

```
x[10:20]
## [1] 59 60 61 62 63 64 65 66 67 68 69
```

Cette commande demande à R de renvoyer les éléments `10:20` du vecteur `x`.

``{r, results = "asis", eval = FALSE}`` Pour changer de rythme, lancez des calculs complexes de sous-ensembles et demandez aux élèves de s'affronter pour trouver le résultat correct en premier. Par exemple :

Créez un vecteur contenant tous les nombres entiers compris entre 51 et 151.

Quel est le résultat de la division du 10^e élément par le 3^e élément ?

Quel est le résultat de l'addition du 70^e élément et du 31^e élément ?

Quelle est la moyenne des éléments compris entre le 40^e et le 60^e ?

Etc.'

...

Exercices

Création de séquences de nombres

1. Utilisez le symbole deux-points pour créer un vecteur de longueur 5 entre une valeur minimale et une valeur maximale de votre choix.
2. Créez un deuxième vecteur de longueur 5 à l'aide de la fonction `seq()`. Utilisez du code pour vérifier

que la longueur de ce vecteur est bien 5.

3. Créez un troisième vecteur de longueur 5 à l'aide de la fonction `rep()` . Utilisez du code pour vérifier que la longueur de ce vecteur est bien 5.
4. Enfin, concaténez les trois vecteurs et vérifiez que la longueur est égale à 15.

Mathématiques vectorielles de base

5. Créez une variable `x` qui est une liste de nombres de taille quelconque. Créez une variable `y` de même longueur.
6. Vérifiez si chaque valeur de `x` est supérieure à chaque valeur de `y`.
7. Vérifiez si la plus petite valeur de `x` est supérieure ou égale à la valeur moyenne de `y` .

Vecteurs et classes d'objets

8. Créez un vecteur contenant au moins un nombre, puis un deuxième vecteur contenant au moins une chaîne de caractères, puis un troisième vecteur contenant au moins une valeur logique. Identifiez la classe des trois vecteurs.
9. Maintenant, concaténez ces trois vecteurs en un quatrième vecteur. Identifiez la classe de ce quatrième vecteur.

Têtes et queues

10. Créez un vecteur contenant au moins 15 valeurs.
11. Affichez les six premières valeurs de ce vecteur à l'aide de la fonction `head()` .
12. Trouvez comment afficher le même résultat sans utiliser de fonction, mais en utilisant vos nouvelles compétences en matière de sous-ensembles de vecteurs. Répliquez maintenant la fonction `tail()` en utilisant ces mêmes compétences. Vous devrez peut-être également appeler la fonction `length()` .

Pointures de chaussures

13. Créez un vecteur appelé `shoes` , qui contient les pointures de chaussures de cinq personnes assises près de vous. Utilisez des commentaires pour savoir à qui correspond chaque pointure.
14. Classez cet ensemble de pointures par ordre croissant.
15. Classez cet ensemble de pointures par ordre décroissant.
16. Utilisez du code pour trouver les deux plus grandes pointures de votre vecteur. N'utilisez pas de sous-ensembles ; écrivez plutôt une ligne de code qui fonctionnerait même si d'autres chaussures étaient ajoutées à votre vecteur.
17. Quelle est la pointure la plus proche de la moyenne de ces pointures ?
18. Utilisez la fonction `which()` pour déterminer à laquelle de vos cinq voisines appartient cette pointure.

Chronologie des baignades

19. Créez maintenant un nouveau vecteur appelé " swim_days ", qui contient le nombre de jours écoulés depuis la dernière baignade de ces cinq personnes (dans n'importe quel plan d'eau ; une estimation du nombre de jours écoulés suffit).
20. Utilisez le code pour demander si quelqu'un est allé nager il y a moins de cinq jours.
21. Parmi vos voisins, lesquels sont allés nager au cours du dernier mois ?
22. Parmi vos voisins, lesquels ne sont pas allés nager au cours du dernier mois ?
23. En moyenne, depuis combien de temps ces personnes ne sont-elles pas allées nager ?

Gérer les valeurs " NA "

24. Créez un vecteur nommé " x " avec les valeurs suivantes : " c(4,7,1,NA,9,2,8) ".
25. Utilisez une fonction pour déterminer si chaque élément de " x " est " NA ".
26. Utilisez une autre fonction pour déterminer quel élément de " x " est " NA ".
27. Écrivez un code qui créera un sous-ensemble de " x " contenant uniquement les valeurs " NA ".
28. Écrivez un code qui créera un sous-ensemble de " x " contenant uniquement les valeurs qui ne sont *pas* " NA ".

Déficits de sommeil

29. Créez maintenant un vecteur appelé " sleep_time " avec le nombre d'heures que vous avez dormi chaque jour au cours de la semaine dernière.
30. Vérifiez si vous avez dormi plus le jour 3 que le jour 7.
31. Calculez le nombre total d'heures de sommeil au cours de la semaine dernière.
32. Calculez le nombre moyen d'heures de sommeil au cours de la semaine dernière.
33. Vérifiez si le nombre total d'heures au cours des 3 premiers jours est inférieur au nombre total d'heures au cours des 4 derniers jours.
34. Créez maintenant un objet nommé " over_under ". Il doit s'agir de la différence entre le nombre d'heures que vous avez dormi chaque nuit et 8 heures (par exemple, 1,5 signifie que vous avez dormi 9,5 heures et -2 signifie que vous avez dormi 8 heures).
35. Écrivez un code pour utiliser " over_under " afin de calculer votre déficit/excédent de sommeil cette semaine (c'est-à-dire le nombre total d'heures en plus/en moins par rapport au nombre d'heures de sommeil que vous auriez eu si vous aviez dormi 8 heures chaque nuit).
36. Écrivez un code pour obtenir le nombre minimum d'heures que vous avez dormi cette semaine.
37. Écrivez un code pour calculer le nombre d'heures de sommeil que vous auriez eu si vous aviez dormi le nombre minimum d'heures chaque nuit.

38. Écrivez un code pour calculer la moyenne des heures de sommeil que vous avez eues du 3e au 6e jour de la semaine.
39. Écrivez un code pour calculer combien d'heures de sommeil vous auriez en un an si vous dormiez chaque nuit autant que la moyenne de vos heures de sommeil entre le 3e et le 6e jour de la semaine.
40. Écrivez un code pour calculer le nombre d'heures de sommeil par an d'une personne qui dort 8 heures par nuit.
41. Combien d'heures de plus ou de moins que les 8 heures par nuit dormez-vous par an, en supposant que vous dormiez chaque nuit en moyenne autant que vous avez dormi le premier et le dernier jour de cette semaine ?
42. Quel est votre déficit de sommeil total pour la semaine dernière ?
43. Combien d'heures supplémentaires par nuit, en moyenne, devez-vous dormir pendant le reste du mois pour que, à la fin du mois, votre déficit de sommeil soit nul ?